

Arquitectura de Datos

Tema 3: Redis

Bases de datos clave-valor en memoria

Curso académico 2026/2027



T3.3 Caché y patrones de uso



Resultados de aprendizaje

- a. Comprender los conceptos de caché, TTL y expulsión de datos
- b. Conocer las estrategias de caché: Cache-Aside, Write-Through, Write-Behind, Read-Through
- c. Entender el problema del thundering herd y cómo mitigarlo con SETNX
- d. Aplicar patrones prácticos: sesiones, rate limiting, rankings con Sorted Sets

Índice de contenidos

T3.3.1 Concepto de caché	T3.3.4 Políticas de expulsión (Eviction)
T3.3.2 Estrategias de caché	T3.3.5 Patrones prácticos de caché
T3.3.3 Invalidación de caché	

T3.3.1 Concepto de caché

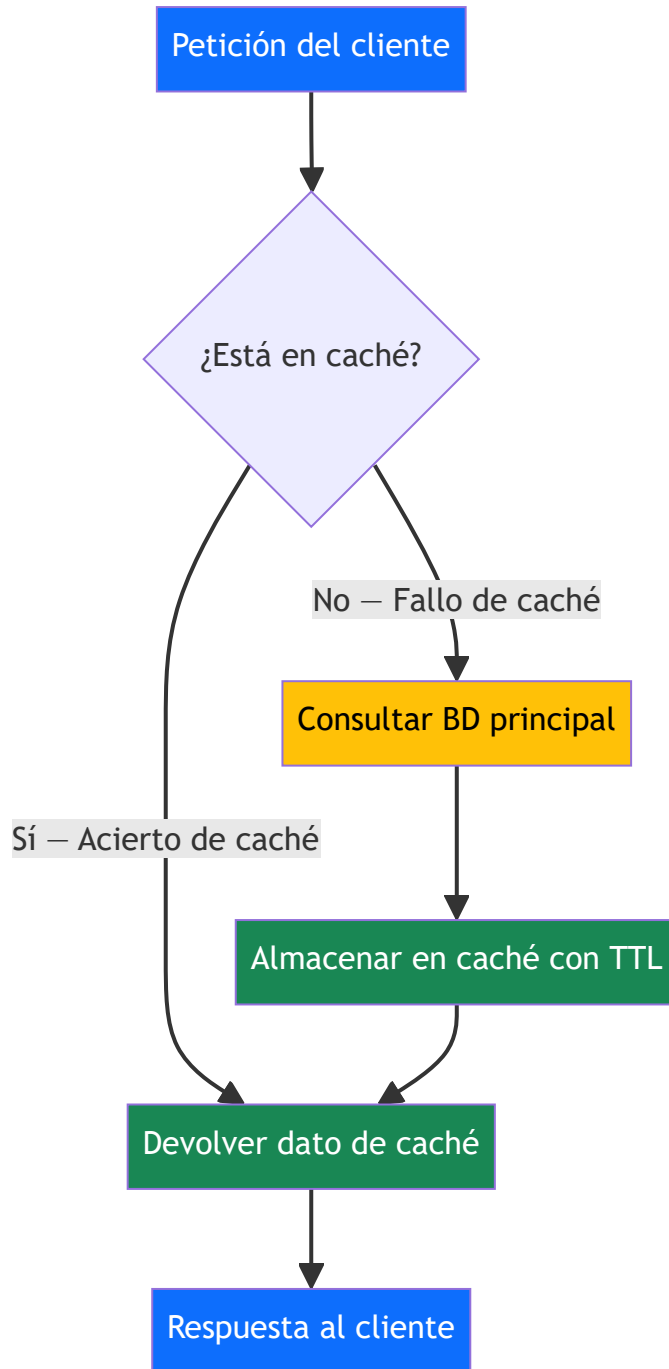
Una **caché** es un almacén de datos de acceso rápido que almacena copias de datos que se consultan frecuentemente. Su objetivo es reducir la latencia y la carga sobre el sistema de almacenamiento principal.

Latencia comparada

Medio	Latencia típica	Factor relativo	Ejemplo
CPU L1 cache	~0,5 ns	1×	Registro del procesador
CPU L2 cache	~7 ns	14×	Caché del núcleo
RAM (Redis)	~100 ns (0,1 μs)	200×	Almacenamiento principal
SSD	~100 μs (0,1 ms)	200 000×	Disco estado sólido
Red (localhost)	~500 μs (0,5 ms)	1 000 000×	Mismo servidor
Red (cross-region)	~50–100 ms	100 000 000×	US-EU

Como se ve, Redis en memoria es **1.000 veces más rápido que SSD** y **100 millones de veces más rápido que una petición cross-region**. Por eso Redis es tan efectivo como caché.

Flujo de una petición con caché



Métricas clave de caché

- **Hit rate (tasa de acierto):** Porcentaje de peticiones que se resuelven desde caché. Un hit rate del 95% significa que solo el 5% de las peticiones llegan a la BD.
- **Miss rate (tasa de fallo):** Porcentaje de peticiones que necesitan consultar la BD.
- **TTL medio:** Tiempo promedio que un dato permanece en caché antes de expirar.

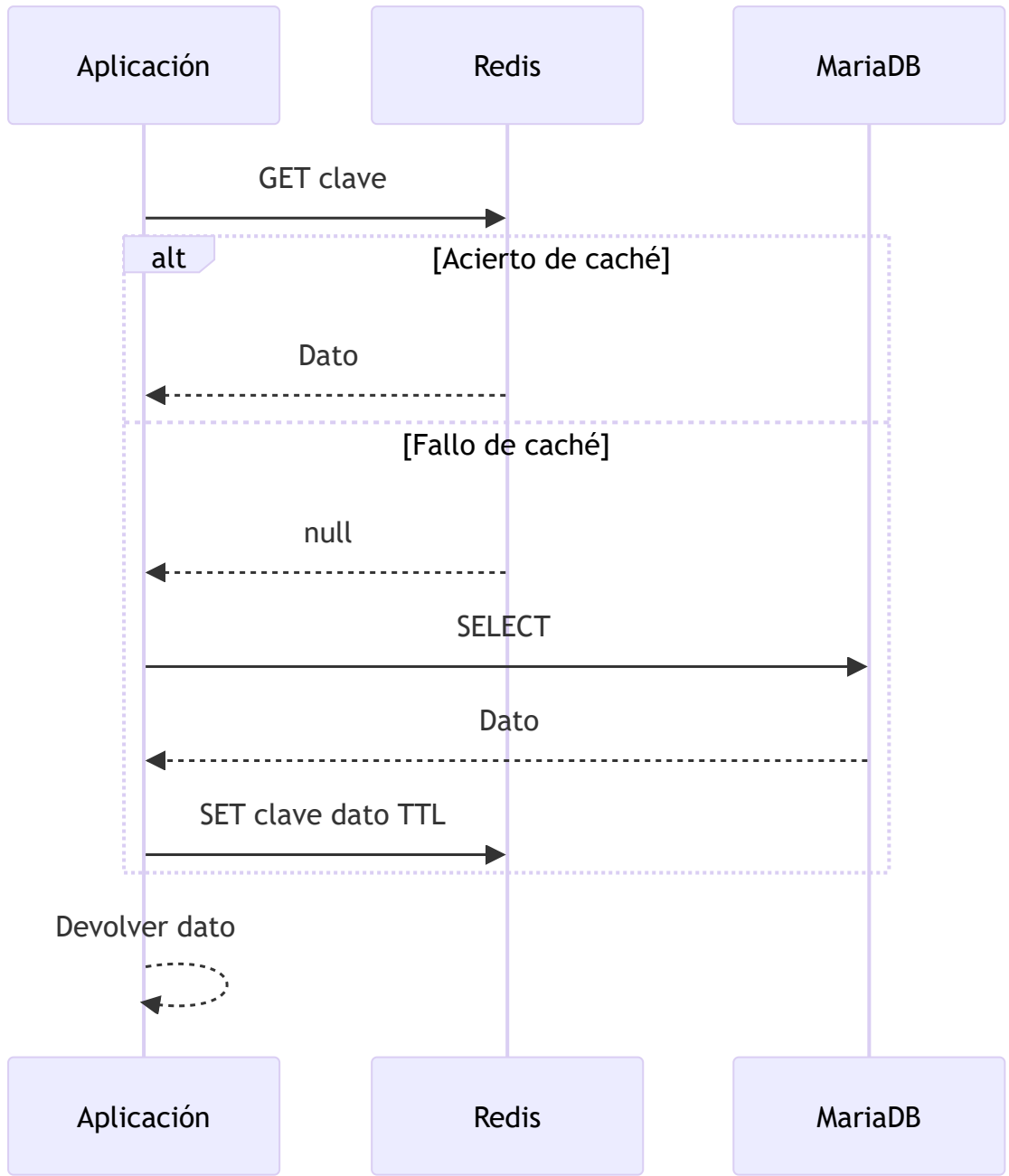
Netflix EVCache consigue latencia sub-milisegundo para el 99% de peticiones de catálogo, reduciendo llamadas a Cassandra en un 95%. Su hit rate promedio supera el 99%, lo que significa que solo 1 de cada 100 peticiones necesita ir a la base de datos.

Resumen T3.3.1: Redis en RAM es 1.000x más rápido que SSD. Hit rate del 95% significa que solo el 5% de peticiones llega a la BD. El objetivo de la caché es maximizar aciertos y minimizar consultas a la BD.

T3.3.2 Estrategias de caché

Cache-Aside (Lazy Loading)

La estrategia más común. La aplicación es responsable de gestionar la caché:



Ventaja: Solo se carga lo que se necesita. **Desventaja:** Tres accesos en caso de fallo (famoso *cache miss penalty*).

Analogía — La farmacia: Imagina que vas a una farmacia a comprar un medicamento habitual.

- **La estantería de la farmacia** es la caché: acceso rápido, pocos productos, pero siempre los más demandados.
- **El almacén del distribuidor** es la base de datos: tiene todo, pero tarda más en llegar.

El proceso es igual que cache-aside:

1. **Hit:** El farmacéutico busca en la estantería → lo tiene → te lo da en 5 segundos.

2. **Miss:** No lo tiene en estantería → llama al distribuidor (DB) → lo recibe → lo pone en estantería (SET) con una fecha de caducidad (TTL) → te lo da.
3. **Próxima vez:** Otro cliente pide lo mismo → ya está en estantería (hit).

¿Por qué no guardar **todo** en la estantería? Porque es **pequeña y cara** (RAM). Solo guardas lo que se consulta frecuentemente. Si un medicamento lleva 6 meses sin venderse, lo retiren (evicción por TTL o LRU).

Ejemplo Cache-Aside en C#

C# – Cache-Aside con StackExchange.Redis

```
using StackExchange.Redis;
using System.Text.Json;

var mux = await ConnectionMultiplexer.ConnectAsync("localhost");
var db = mux.GetDatabase();

async Task<Producto?> GetProductoAsync(int id)
{
    var key = $"producto:{id}";
    var cached = await db.StringGetAsync(key);

    if (!cached.IsNullOrEmpty)
        return JsonSerializer.Deserialize<Producto>(cached!);

    // Cache miss – consultar BD
    var producto = await _db.Productos.FindAsync(id);
    if (producto != null)
    {
        var json = JsonSerializer.Serialize(producto);
        await db.StringSetAsync(key, json, TimeSpan.FromMinutes(30));
    }
    return producto;
}
```

Ejemplo Cache-Aside en Python

Python – Cache-Aside con redis-py

```
import redis
import json

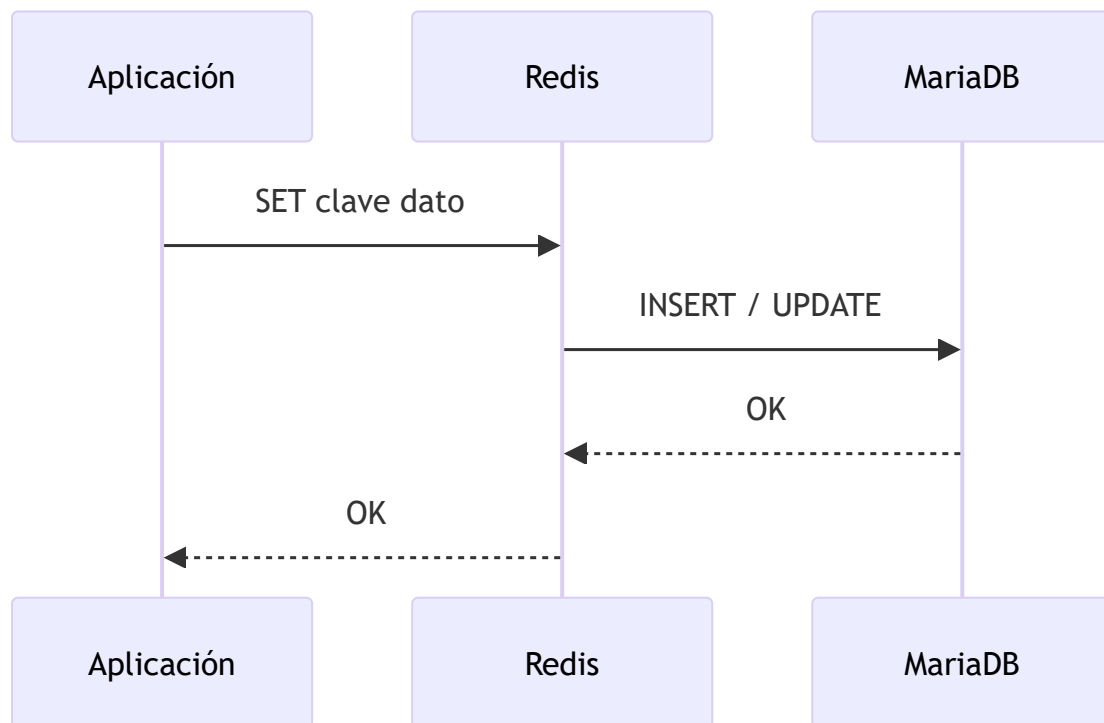
r = redis.Redis(host='localhost', port=6379, decode_responses=True)

def get_producto(producto_id: int) -> dict | None:
    key = f"producto:{producto_id}"
    cached = r.get(key)

    if cached:
        return json.loads(cached)

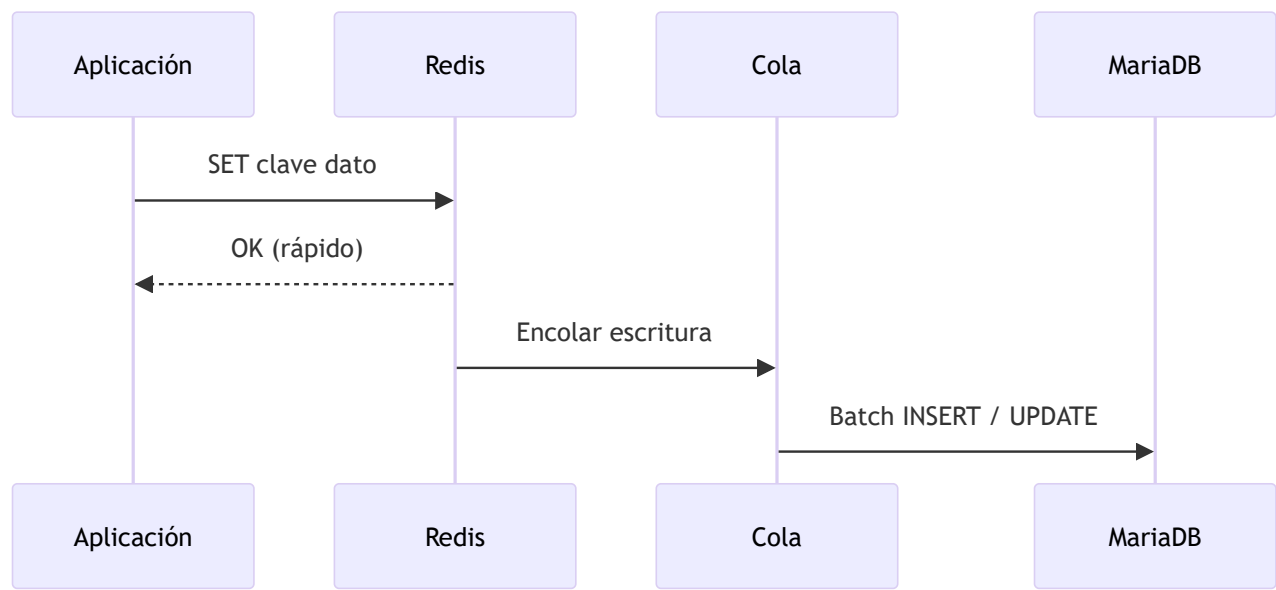
    # Cache miss – consultar BD
    producto = db.query("SELECT * FROM productos WHERE id = %s", producto_id)
    if producto:
        r.setex(key, 1800, json.dumps(producto)) # TTL 30 min
    return producto
```

Write-Through



Ventaja: Consistencia garantizada (caché y BD siempre sincronizadas). **Desventaja:** Escritura más lenta (dos operaciones).

Write-Behind (Write-Back)



Ventaja: Escritura muy rápida. **Desventaja:** Riesgo de pérdida de datos si falla la cola.

T3.3.2.4 Read-Through (lectura a través de caché)

La caché gestiona la carga desde la BD. La aplicación solo consulta la caché; si no tiene el dato, es la caché misma quien lo carga desde la base de datos:

Read-Through – La caché carga el dato

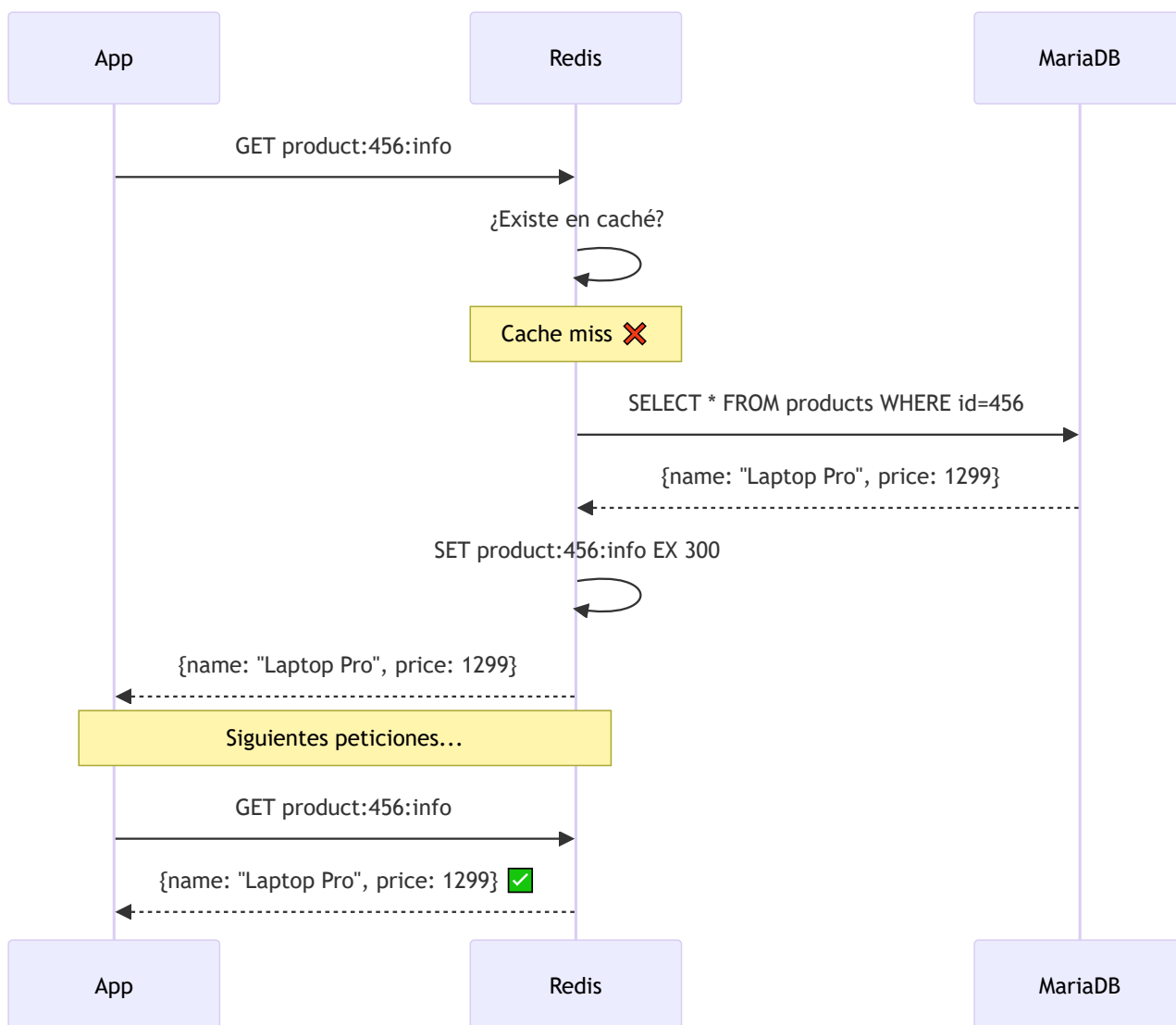
```
# La aplicación solo habla con Redis
# Redis carga desde MariaDB si no encuentra el dato

# 1. Aplicación consulta Redis
GET product:456:info
# → (nil) (cache miss)

# 2. Redis internamente consulta MariaDB
# SELECT * FROM products WHERE id=456
# → {name: "Laptop Pro", price: 1299}

# 3. Redis almacena y devuelve el resultado
# SET product:456:info '{"name":"Laptop Pro","price":1299}' EX 300
# → '{"name": "Laptop Pro", "price": 1299}'

# 4. Sigüientes consultas son cache hit
GET product:456:info
# → '{"name": "Laptop Pro", "price": 1299}' (0.5ms vs 50ms)
```



Read-Through vs Cache-Aside: La diferencia es quién carga el dato desde la BD. En Cache-Aside lo hace la aplicación. En Read-Through lo hace la propia caché. Read-Through simplifica el código de la aplicación pero requiere una capa de caché inteligente (Redis no lo hace nativamente, se implementa con Lua scripts o módulos).

Cuándo usar Read-Through: Cuando quieres simplificar la lógica de la aplicación y delegar la gestión de caché a una capa intermedia. En Redis se implementa con Lua scripts que ejecutan `GET` + fallback a BD en un solo roundtrip.

Instagram usa Write-Behind para likes: incrementa en Redis al instante (`INCR post_likes:{post_id}`) y persiste en Cassandra de forma asíncrona. El usuario ve el like instantáneamente, y Cassandra se actualiza unos milisegundos después.

Resumen T3.3.2: Cache-Aside (80% de casos), Write-Through (consistencia fuerte), Write-Behind (máxima velocidad), Read-Through (caché gestiona la carga). Elige según necesidad de consistencia y rendimiento.

¿Qué estrategia de caché uso?

Estrategia	Consistencia	Rendimiento escritura	Complejidad	Cuándo usar
Cache-Aside	Eventual	Media	Baja	Caché general, datos que se leen más de lo que se escriben
Write-Through	Fuerte	Baja (2 writes)	Media	Datos críticos que necesitan consistencia inmediata
Write-Behind	Eventual	Muy alta (1 write)	Alta	Métricas, likes, eventos de alta frecuencia donde la pérdida temporal es aceptable
Read-Through	Eventual	Media	Media	Cuando la caché gestiona la carga automáticamente (no la aplicación)

En la práctica: El 80% de los casos usan **Cache-Aside** por su simplicidad. Write-Behind es ideal para métricas (Instagram likes). Write-Through es raro en la práctica porque dobla el coste de escritura.

T3.3.3 Invalidación de caché

La invalidación de caché es uno de los problemas más difíciles en computer science. El objetivo es mantener la caché sincronizada con la base de datos.

Expiración por tiempo (TTL)

Configuración de TTL

```
# TTL en segundos
SET cache:product:456 "iPhone" EX 300      # 5 minutos

# TTL en milisegundos
SET cache:session:abc "data" PX 1800000    # 30 minutos

# Añadir TTL a clave existente
EXPIRE cache:product:456 600               # 10 minutos

# Verificar TTL restante
TTL cache:product:456  # -1 = sin expiración, -2 = no existe, >0 = segundos
```

Invalidación por evento

Invalidación cuando cambia la BD

```
# 1. Actualizar MariaDB
# UPDATE products SET price=29.99 WHERE id=456;

# 2. Invalidar caché en Redis
DEL cache:product:456

# 3. Opcionalmente, pre-cargar el nuevo valor
SET cache:product:456 '{"name":"iPhone","price":29.99}' EX 300
```

«La invalidación de caché es uno de los dos problemas difíciles de la informática.» — Phil Karlton

Problemas comunes

- **Caché obsoleta (stale cache):** El dato en caché está desactualizado respecto a la BD. Solución: TTL corto o invalidación por evento.
- **Efecto de multitud (thundering herd):** Cuando expira una clave popular, múltiples clientes consultan la BD simultáneamente, saturándola. Solución: locks o TTL desfasados.
- **Caché rota (cache stampede):** Variante del thundering herd donde muchos clientes intentan regenerar la caché al mismo tiempo.

```
# PROBLEMA: 1000 clientes piden product:456 al mismo tiempo
# El TTL expira → 1000 consultas simultáneas a MariaDB

# SOLUCIÓN 1: Lock con SETNX (solo 1 cliente carga el dato)
# Cliente 1: intenta adquirir el lock
SET lock:product:456 "loading" NX EX 10
# → OK (adquirió el lock)

# Clientes 2-1000: intentan adquirir el lock
SET lock:product:456 "loading" NX EX 10
# → (nil) (no adquirieron el lock)

# Cliente 1: carga el dato desde MariaDB
SELECT * FROM products WHERE id=456
# → {name: "Laptop Pro", price: 1299}

# Cliente 1: almacena el dato y libera el lock
SET product:456:info '{"name":"Laptop Pro","price":1299}' EX 300
DEL lock:product:456

# Clientes 2-1000: ahora encuentran el dato en caché
GET product:456:info
# → {'name': 'Laptop Pro', 'price': 1299} ✓

# SOLUCIÓN 2: TTL desfasado (jitter)
# En lugar de EX 300, usa EX random entre 270-330
SET product:456:info '{"name":"Laptop Pro"}' EX $((RANDOM % 60 + 270))
# Los TTLs de diferentes clientes no expiran todos a la vez
```

SETNX + EX: SET key value NX EX seconds es atómico: solo un cliente obtendrá OK. Los demás esperan o leen de caché (si ya se cargó). El jitter (TTL aleatorio) distribuye la expiración en el tiempo, evitando picos.

Jitter = aleatorización intencional de un valor para evitar sincronización no deseada. En contexto de caché: si 1000 claves tienen TTL=3600, añadir jitter de $\pm 10\%$ (360-3960s) evita que todas expiren al mismo tiempo. Sin jitter, las 1000 claves expiran simultáneamente → 1000 queries a la BD de golpe (thundering herd). Con jitter, las expiraciones se distribuyen a lo largo de ~6 minutos.

Twitter invalida timelines en Redis cuando un usuario borra un tweet, usando eventos de Kafka para notificar a todas las réplicas en tiempo real. El proceso es: 1) Tweet se borra de Cassandra, 2) Evento se publica en Kafka, 3) Todos los Consumers de Kafka borran el tweet de los timelines en Redis.

Resumen T3.3.3: TTL para expiración por tiempo. Invalidación por evento (DEL) cuando cambia la BD. Cuidado con el efecto de multitud (thundering herd) al expirar claves populares.

Invalidación vs Evicción: Son dos problemas diferentes del mismo contexto. La **invalidación** resuelve el problema de **consistencia**: ¿cómo mantengo la caché sincronizada cuando los datos originales cambian? La **evicción** resuelve el problema de **memoria**: ¿qué hago cuando la caché se llena y necesito espacio para nuevos datos? En la práctica, ambos se gestionan juntos: defines un TTL para invalidar por tiempo, y una política de evicción para gestionar la memoria.

T3.3.4 Políticas de expulsión (Eviction)

Cuando la memoria Redis se llena, necesita **expulsar** claves para liberar espacio. Las políticas de expulsión determinan qué claves se eliminan:

Contenido avanzado: Las políticas de evicción son importantes para la producción, pero para entender los conceptos básicos de caché puedes centrarte en `allkeys-lru` (la más habitual) y `noeviction`.

LRU (Least Recently Used): Elimina el dato que hace más tiempo se usó. Siendo `access_time` la última vez que se leyó/escribió la clave, LRU elimina la clave con `access_time` más antiguo. Simple pero puede eliminar datos que se usan regularmente si hubo un pico de tráfico.

LFU (Least Frequently Used): Elimina el dato que menos veces se ha usado en una ventana de tiempo reciente. Redis usa un contador aproximado (no exacto) para evitar overhead de memoria. Más preciso que LRU para cachés con patrones de uso variables.

Política	Descripción	Cuándo usar
<code>noeviction</code>	Error cuando la memoria está llena	Datos críticos donde perder datos es inaceptable. Redis devuelve error en escrituras.
<code>allkeys-lru</code>	Least Recently Used en todas las claves	Caché general de API (cache-aside). No importa qué clave se elimina, solo mantener las más recientes.
<code>volatile-lru</code>	LRU solo en claves con TTL	Sesiones de usuario con TTL. Los datos sin TTL (config permanentes) nunca se eliminan.
<code>allkeys-lfu</code>	Least Frequently Used	Caché de productos donde los más vendidos deben permanecer más tiempo.
<code>volatile-lfu</code>	LFU solo en claves con TTL	Similar a <code>volatile-lru</code> pero por frecuencia.
<code>volatile-ttl</code>	Expulsa claves con TTL más corto	Datos con diferentes niveles de frescura: datos de 5 min vs datos de 1 hora. Elimina los que caducan primero.
<code>allkeys-random</code>	Expulsión aleatoria	Cuando todos los datos son igual de importantes.
<code>volatile-random</code>	Aleatorio solo en claves con TTL	Similar a <code>allkeys-random</code> pero con TTL.

«Lo que mides, lo gestionas.» — Peter Drucker. Monitoriza siempre la tasa de aciertos de caché (hit rate). Si cae por debajo del 80%, es hora de reconsiderar la estrategia de caché.

Producción: Configura siempre `maxmemory-policy allkeys-lru` en producción. Sin esta configuración, Redis devolverá errores cuando la memoria se llene en lugar de expulsar claves antiguas.

Configuración de políticas de expulsión

```
# En redis.conf
maxmemory 2gb
maxmemory-policy allkeys-lru

# O en tiempo de ejecución
CONFIG SET maxmemory 2gb
CONFIG SET maxmemory-policy allkeys-lru

# Verificar configuración actual
CONFIG GET maxmemory
CONFIG GET maxmemory-policy
```

GitHub usa `volatile-lru` para expulsar sesiones antiguas mientras mantiene en caché las claves de rate limiting que no tienen TTL. Las sesiones tienen TTL (30 min), pero los contadores de rate limiting no (se resetean por timestamp), por lo que `volatile-lru` solo expulsa las sesiones.

Resumen T3.3.4: `allkeys-lru` es la política más habitual. `volatile-lru` solo expulsa claves con TTL. `noeviction` genera error si la memoria se llena. Configura `maxmemory` según tu presupuesto de RAM.

T3.3.5 Patrones prácticos de caché

Patrón 1: Sesión de usuario

Gestión de sesiones

```
# Crear sesión tras Login
SET session:abc123:userId "1000" EX 1800
SET session:abc123:role "admin" EX 1800
SET session:abc123:lastAccess "2026-09-15T10:30:00" EX 1800

# Verificar sesión en cada petición
GET session:abc123:userId

# Renovar sesión (sliding expiration)
EXPIRE session:abc123:userId 1800

# Cerrar sesión (logout)
DEL session:abc123:userId
DEL session:abc123:role
DEL session:abc123:lastAccess
```

Patrón 2: Caché de producto (Cache-Aside)

Cache-Aside para productos

```
# Flujo de lectura (pseudocódigo de la aplicación):
# 1. value = redis.GET("product:456:info")
# 2. if value == null:
# 3.     data = db.query("SELECT * FROM products WHERE id=456")
# 4.     redis.SET("product:456:info", data, EX=300)
# 5.     return data
# 6. return value

# Invalidar cuando se actualiza
# UPDATE products SET price=29.99 WHERE id=456;
DEL product:456:info
```

Patrón 3: Rate limiting con ventana deslizante

Rate limiting con Sorted Sets

```
# Registrar petición
ZADD ratelimit:user:1000 1726400000 "req-001"

# Eliminar peticiones fuera de la ventana (última hora)
ZREMRANGEBYSCORE ratelimit:user:1000 0 1726396400

# Contar peticiones en la ventana actual
ZCARD ratelimit:user:1000

# Si ZCARD > 100 → rechazar la petición (HTTP 429)
```

Twitch: Como vimos en T3.1, Twitch Plays Pokémon recibió 1,2 millones de mensajes en 24 horas y la solución fue shard por canal de chat.

Resumen T3.3.5: Sesiones con TTL. Cache-Aside para productos. Rate limiting con Sorted Sets y ventana deslizante. Patrones que cubren el 90% de casos de uso reales.

Caso real: GitHub — Rate Limiting con Sorted Sets

GitHub necesita rate limiting con **5.000 peticiones/hora** por usuario autenticado. La solución usa Redis con Sorted Sets para garantizar atomicidad sin necesidad de scripts de servidor.

¿Por qué Sorted Sets y no INCR?

INCR + EXPIRE son dos comandos separados y no son atómicos juntos. Si el cliente muere entre ambos, la clave queda sin TTL. ZADD es atómico por diseño: cada petición se registra de forma segura.

Implementación en C# 14:

C# – Rate limiting con Sorted Sets

```
var key = $"ratelimit:user:{userId}";
var now = DateTimeOffset.UtcNow.ToUnixTimeSeconds();
var windowStart = now - 3600; // ventana de 1 hora

// ZADD es atómico: registra cada petición
await db.SortedSetAddAsync(key, now, $"{now}:{Guid.NewGuid()}");

// Eliminar peticiones fuera de ventana
await db.SortedSetRemoveRangeByScoreAsync(key, 0, windowStart);

// Contar peticiones en ventana actual
var count = await db.SortedSetLengthAsync(key);

if (count > 5000)
{
    return Results.StatusCode(429); // Too Many Requests
}
```

Arquitectura en producción:

- **Sharding client-side:** cada instancia de la aplicación elige qué clúster Redis usar.
- **Primary + réplicas:** primary para escrituras, réplicas para lecturas de verificación.
- **Feature flags:** migración gradual de un sistema de rate limiting a otro.

Lección: En producción, la atomicidad de las operaciones Redis es crítica. ZADD garantiza que dos peticiones concurrentes del mismo usuario nunca se pierdan, sin necesidad de transacciones complejas.

Caso real: Netflix — EVCache

Netflix usa **EVCache**, un sistema de caché distribuido basado en Memcached/Redis, para servir contenido a más de 230 millones de suscriptores en todo el mundo.

Casos de uso:

Caso	Descripción
Lookaside Cache	Aplicación → EVCache → Cassandra (fallback). Consulta caché primero; si falla, consulta Cassandra.
Datos transitorios	Sesiones de reproducción entre microservicios. Se borran automáticamente al cerrar sesión.
Pre-cómputo	Homepage personalizada calculada cada noche, servida desde caché.
Alto volumen	Strings de UI e idiomas. Alta disponibilidad, baja latencia.

Arquitectura:

- **Multi-región:** caché por continente para minimizar latencia.
- **Fallback automático:** si la caché falla, lectura directa de Cassandra.
- **Latencia:** sub-milisegundo para el 99% de peticiones de catálogo.

Lección: Redis no funciona solo. En sistemas globales como Netflix, Redis/evcache se integra con bases de datos como Cassandra en una arquitectura de múltiples capas con fallback automático.

Cuando lo pongas en práctica

- Siempre define TTL — una caché sin TTL es una bomba de memoria que acabará expulsando datos útiles.
- Mide tu hit rate con INFO keyspace antes de optimizar; no asumas que la caché funciona bien.
- Para Write-Behind, considera Redis Streams como cola de escritura asíncrona a la BD.
- En caché de sesiones, usa SETEX (SET + EXPIRE atómico) para evitar claves sin TTL por errores de la aplicación.
- Para datos críticos (precio, stock), invalidación manual con DEL es más fiable que depender del TTL.

Preguntas para reflexionar

Pregunta 1

Si implementas Cache-Aside y el TTL expira justo cuando un usuario consulta un producto, ¿qué problema aparece? ¿Cómo lo mitigarías?

Pregunta 2

¿Por qué no se debería usar SETNX para rate limiting en una aplicación con miles de usuarios concurrentes? ¿Qué alternativa usarías?

Pregunta 3

Si tuvieras que elegir entre cachear los datos más recientes (LRU) o los más consultados (LFU) para una aplicación de noticias, ¿cuál elegirías? Justifica.



Arquitectura de Datos

4.º Curso, Grado en Ingeniería Informática

Curso académico 2026/2027

Human Language and Accessibility Technologies Group (HULAT).

Departamento de Informática

Universidad Carlos III de Madrid